Implementación de módulos independientes enfocados en tareas específicas de estadística descriptiva

Azuara Yáñez Cecil Quimey
Universidad de Guanajuato
División de Ingenierías
Campus Irapuato - Salamanca
cq.azuarayanez@ugto.mx

***Resumen*—Este reporte documenta el diseño, la estructuración y la implementación de un paquete jerárquico modular en Python (estadisitica_analitica_lib) orientado al análisis estadístico de datos agrupados. El sistema fue desarrollado siguiendo el patrón de diseño de doble contexto (__name__ == ”__main__”), permitiendo su funcionamiento tanto como una librería reutilizable como un script de auditoría autonoma. La arquitectura del software se dividió en cuatro submódulos especializados: distribución de frecuencias (construcción de clases, marcas de clase y frecuencias acumuladas/relativas), medidas de tendencia central (promedios aritmético, geométrico, armónico, RCM, mediana, moda y cuantiles), medidas de dispersión (rangos, desviación media, varianza, desviación estándar y coeficiente de variación) y análisis de forma (momentos crudos y centrales, sesgo de Fisher-Pearson y exceso de curtosis). La validación del sistema se llevó a cabo a través de un script principal (main.py) que ejecutó una auditoría automatizada sobre series de 100 observaciones continuas (valores flotantes) y discretas (valores enteros).**

Los resultados se presentaron en consola mediante una función de visualización tabular formateada con f-strings. Los resultados obtenidos demostraron la precisión matemática de las fórmulas estadisticas implementadas y la efectividad de la estructura modular propuestas.

I. Introducción

En la estadística descriptiva, el tratamiento de grandes conjuntos de datos cuantitativos (tanto discretos como continuos) requiere de técnicas de condensación e interpretación eficientes. Cuando el volumen de observaciones resulta elevado, la organización de los datos en distribuciones de frecuencias por intervalos de clase se convierte en la estrategia fundamental para identificar patrones conceptuales de centralización, variabilidad y comportamiento geométrico de la muestra.

El cálculo de métricas en datos agrupados exige un rigor formal diferente al de los datos crudos o no agrupados. Métricas centrales como la media aritmética, geométrica, armónica y la raíz cuadrada media (RCM) asumen que los valores de cada clase concuerdan con su respectiva marca de clase x_j. Por su parte, el cálculo de las medidas posicionales (mediana, moda y cuantiles) demanda métodos de interpolación lineal sobre los límites reales de la distribución acumulada. Asimismo, la evaluación integral de la variable requiere cuantificar su dispersión (desviación media, varianza, desviación estándar y coeficiente de variación) y caracterizar su geometría analítica mediante momentos estadísticos, midiendo así el grado de asimetría (sesgo) y apuntamiento (curtosis).

En el ámbito de la ciencia de datos y la programación orientada a objetos, el desarrollo de software estadístico no solo requiere exactitud en los algoritmos matemáticos, sino también una arquitectura de software limpia y modular. El uso de paquetes y submódulos jerárquicos facilita la mantenibilidad del código, la reutilización de componentes y la abstracción de procesos. Además, la implementación del patrón de diseño de doble contexto (`if __name__ == "__main__":`) constituye una buena práctica esencial, permitiendo que cada módulo actúe de forma autónoma como un script de auditoría o de manera integrada como una librería cliente importable.

La implementación de estructuras de control iterativas y manipulación de listas se realizó utilizando la sintaxis estándar del lenguaje (González Duque, 2014).

Así mismo, como señala González Duque (2014), la creación de paquetes mediante el archivo __init__.py y el bloque if __name__ == '__main__': permite organizar el código en componentes reutilizables e independientes.

I-A. Objetivos específicos:

Diseñar, estructurar e implementar un paquete jerárquico modular en Python (`estadisitica_analitica_lib`) orientado al análisis estadístico univariante de datos agrupados.

1. Desarrollar un módulo (`Frecuencias.py`) para determinar la cantidad de clases (Regla de Sturges), límites reales, marcas de clase y frecuencias absolutas,

acumuladas y relativas.

2. Implementar un módulo (`Tendencias.py`) para calcular promedios centrales (aritmético, geométrico, armónico, RCM) y medidas de posición interpoladas (mediana, moda y cuantiles).

3. Crear un módulo (`Dispersion.py`) que evalúe la variabilidad absoluta y relativa mediante rangos, desviación media, varianza, desviación estándar y coeficiente de variación.

4. Programar un módulo (`Forma.py`) para obtener momentos crudos, centrales y adimensionales, evaluando el sesgo (Fisher-Pearson, Pearson) y la curtosis.

5. Configurar un script principal (`main.py`) que ejecute una auditoría automatizada sobre series sintéticas de 100 observaciones continuas y discretas, dando como resultado un formato de tabla limpio en consola mediante f-strings.

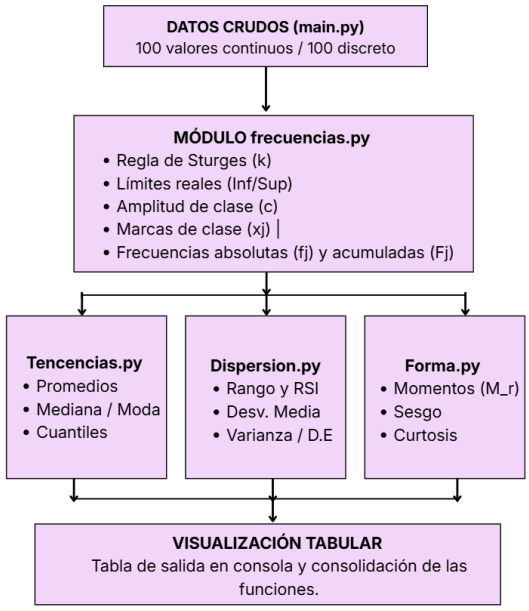


Figura 1. Diagrama de bloques del flujo de procesamiento entre módulos del sistema.

II. Metodología

La metodología adoptada en este trabajo se basa en el desarrollo de un paquete jerárquico modular en Python denominado `estadisitica_analitica_lib`, diseñado específicamente para procesar arreglos de datos univariantes mediante agrupamiento en intervalos de clase. El flujo de trabajo del sistema se dividió en fases secuenciales que abarcan desde la recepción de los datos crudos hasta el despliegue de auditorías tabuladas.

A continuación se muestra un diagrama que presenta el flujo del programa.

II-A. Estructura Jerárquica y Orden de Ejecución

El proyecto se estructuró bajo una arquitectura modular de archivos organizados dentro del espacio de trabajo principal `Modulos_estadistica_datos_agrupados/`. La distribución física y la secuencia lógica de invocación entre componentes se definieron en el siguiente orden:

1. `estadisitica_analitica_lib/__init__.py`: Archivo especial de inicialización que define el directorio como un paquete importable en Python, habilitando la exportación de funciones hacia scripts externos.

2. `estadisitica_analitica_lib/Frecuencias.py`: Submódulo primario y punto de partida del flujo de cálculo. Encargado de agrupar la muestra cruda, determinar la cantidad de intervalos mediante Sturges y construir las marcas de clase (x_j) junto con las frecuencias absolutas (f_j) y acumuladas (F_j).

3. `estadisitica_analitica_lib/Tendencias.py`: Submódulo analítico de segundo nivel. Consume la salida de `Frecuencias.py` para computar las medidas de centralización (μ, G, H, RCM) y las interpolaciones lineales de posición (Mediana, Moda y Cuantiles).

4. `estadisitica_analitica_lib/Dispersion.py`: Submódulo analítico que toma los resultados de frecuencias y la media aritmética de `Tendencias.py` para calcular las métricas de variabilidad muestral (Rangos, Desviación Media, Varianza, Desviación Estándar y CV).

5. `estadisitica_analitica_lib/Forma.py`: Submódulo analítico final que recibe los parámetros de los módulos previos para determinar la geometría de la masa de datos (Momentos crudos y centrales, Sesgo de Fisher-Pearson y Exceso de Curtosis).

6. `main.py`: Script cliente principal situado en la raíz del proyecto. Coordina la ejecución general invocando secuencialmente cada submódulo, gestiona la captura o generación sintética de datos y consolida la tabla de salida formateada en consola.

La organización de la tarea sigue una jerarquía modular estricta dentro del directorio raíz `Modulos_estadistica_datos_agrupados/`. A

continuación se muestra el árbol de archivos del sistema:

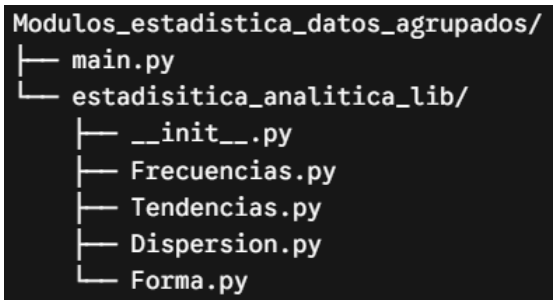


Figura 2. Orden jerárquico modular.

II-B. Estructura Modular

El sistema se organizó bajo una estructura de directorios estricta dentro del espacio de trabajo `Modulos_estadistica_datos_agrupados/`, conteniendo el archivo principal `main.py` y el paquete estadisitica_analitica_lib/ con su correspondiente archivo de inicialización `__init__.py`.

Para garantizar la reutilización del código, cada submódulo implementó el patrón de diseño de doble contexto (`if __name__ == "__main__":`):

1. **Librería cliente:** Permite que `main.py` o scripts externos importen las funciones para procesar conjuntos de datos en producción.

2. **Auditoría autónoma:** Permite ejecutar cada archivo .py de manera independiente desde la consola para auditar y verificar el correcto funcionamiento matemático de sus algoritmos mediante datos de prueba.

II-C. Formulación Matemática

II-C1. Módulo de Distribución de Frecuencias (`Frecuencias.py`): Encargado de agrupar los datos crudos en intervalos de clase mediante la Regla de Sturges para determinar el total de clases k.

Regla de Sturges (Número de clases k):

$$k = 1 + 3{,}322 \cdot \log_{10}(N)$$

A partir del rango total $R = \text{máx(datos)} - \text{mín(datos)}$, se calcula la amplitud del intervalo c.

Amplitud del intervalo (c):

$$c = \frac{R}{k}$$

Las marcas de clase x_j, que representan el punto medio de cada intervalo con límites reales inferiores (L_{inf}) y superiores (L_{sup}), se obtienen con:

Marca de clase (x_j):

$$x_j = \frac{L_{inf} + L_{sup}}{2}$$

Se computan secuencialmente las frecuencias absolutas (f_j), acumuladas (F_j), relativas ($h_j = f_j/N$) y relativas acumuladas (H_j).

II-C2. Módulo de Medidas de Tendencia Central (`Tendencias.py`): Utiliza la marca de clase x_j y la frecuencia f_j obtenidas previamente para calcular las métricas de centralización.

Media Aritmética para datos agrupados (μ):

$$\mu = \frac{\sum_{j=1}^{K} f_j x_j}{N}$$

Media Geométrica: Implementada mediante aproximación logarítmica para evitar desbordamientos numéricos.
(G):

$$G = 10^{\left(\frac{1}{N}\sum_{j=1}^{K} f_j \log_{10}(x_j)\right)}$$

Media Armónica (H):

$$H = \frac{N}{\sum_{j=1}^{K} \frac{f_j}{x_j}}$$

Raíz Cuadrada Media (RCM):

$$\text{RCM} = \sqrt{\frac{\sum_{j=1}^{K} f_j \cdot x_j^2}{N}}$$

Mediana (Interpolación Lineal): Localizada en la primera clase cuya frecuencia acumulada satisface $F_j \geq \frac{N}{2}$, calculada mediante interpolación lineal.

$$\text{Mediana} = L_i + \left[\frac{\frac{N}{2} - F_{prev}}{f_{mediana}}\right] \cdot c$$

Moda:Determinada a partir de la clase modal (mayor f_j) mediante las diferencias de frecuencia contiguas $\Delta_1 = f_{modal} - f_{modal-1}$ y $\Delta_2 = f_{modal} - f_{modal+1}$

$$\text{Moda} = L_i + \left[\frac{\Delta_1}{\Delta_1 + \Delta_2}\right] \cdot c$$

Cuantiles (Q_k): Formulación general para cuartiles ($q = 4$), deciles ($q = 10$) y percentiles ($q = 100$) según la posición $P = \frac{k \cdot N}{q}$.

$$Q_k = L_i + \left[\frac{\frac{k \cdot N}{q} - F_{prev}}{f_{cuantil}}\right] \cdot c$$

II-C3. Módulo de Medidas de Dispersión (`Dispersion.py`): Cuantifica la variabilidad de las observaciones respecto a la media aritmética: Rango Muestral: Obtenido a partir de la diferencia entre la última y primera marca de clase ($x_K - x_1$).

Rango Semi-Intercuartílico (RSI):

$$\text{RSI} = \frac{Q_3 - Q_1}{2}$$

Desviación Media

(D_m):

$$D_m = \frac{1}{N} \sum_{j=1}^{K} f_j |x_j - \mu|$$

Varianza (σ^2) y Desviación Estándar (σ):

$$\sigma^2 = \frac{\sum_{j=1}^{K} f_j (x_j - \mu)^2}{N}, \quad \sigma = \sqrt{\sigma^2}$$

Coeficiente de Variación (CV): Expresado en términos porcentuales para evaluar la dispersión relativa.

$$CV = \left(\frac{\sigma}{\mu} \right) \times 100\,\%$$

II-C4. Módulo de Forma (`Forma.py`): Describe las características geométricas de la distribución de frecuencias.

Momento Crudo de orden r ($\overline{x}^r$): Evaluado respecto al origen:

$$\overline{x}^r = \frac{1}{N} \sum_{j=1}^{K} f_j \cdot x_j^r$$

Momento Central de orden r (m_r): Evaluado alrededor de la media μ:

$$m_r = \frac{1}{N} \sum_{j=1}^{K} f_j \cdot (x_j - \mu)^r$$

Sesgo de Fisher-Pearson (γ_1):

$$\gamma_1 = \frac{m_3}{\sigma^3}$$

Exceso de Curtosis: Medida de apuntamiento estandarizada restando 3 respecto a la distribución normal gaussiana.

$$\kappa = \frac{m_4}{\sigma^4} - 3$$

II-C5. Visualización de Resultados en Script Principal (`main.py`): El script cliente `main.py` genera arreglos sintéticos de auditoría (100 números flotantes continuos y 100 enteros discretos) empleando la librería random. Tras la selección interactiva por parte del usuario, se consolida un diccionario estructurado de resultados que es procesado por la función `Tabla_resultados` (resumen). Dicha función imprime las métricas en la terminal utilizando alineación dinámica con f-strings y caracteres de caja (+, -, —), garantizando un despliegue ordenado y de fácil lectura.

II-D. Entorno de Desarrollo y Estructura del Código

El paquete fue programado utilizando el entorno de desarrollo integrado (IDE) **Visual Studio Code**, aprovechando sus herramientas de depuración y gestión de terminal integrada para la ejecución de scripts.

La arquitectura modular y la sintaxis del código se diseñaron siguiendo las buenas prácticas de programación estructurada y organización de paquetes propuestas por González Duque (2014) en su obra *Python para todos*, así como lo visto y ejercicios hechos durante el curso de ”Fundamentos del Análisis de Datos”. Específicamente, se aplicaron los principios de:

1. **Encapsulamiento en submódulos:** División del código en archivos `.py` independientes dentro del directorio `estadisitica_analitica_lib/`.

2. **Reutilización mediante importación jerárquica:** Declaración del archivo especial `__init__.py` para la construcción formal del paquete.

3. **Control de ejecución con el bloque `__name__`:** Implementación del patrón de doble contexto para garantizar que cada módulo funcione tanto como librería cliente como script de prueba autónomo.

II-E. Algoritmos de Búsqueda e Interpolación Manual

Para cumplir con el requerimiento de lógica independiente sin librerías externas de terceros, los procesos de localización de clases características se implementaron mediante estructuras de control nativas:

1. **Búsqueda de la Clase Mediana:** Se realiza un recorrido iterativo `for` sobre el arreglo de frecuencias acumuladas F_j. En la primera iteración donde $F_j \geq N/2$, se extrae el índice j, guardando la frecuencia acumulada anterior F_{j-1} mediante un condicional simple para evitar desbordamientos de índice cuando $j = 0$.

2. **Búsqueda de la Clase Modal:** Se implementó un algoritmo de búsqueda del valor máximo sobre la lista de frecuencias absolutas f_j, identificando la posición del

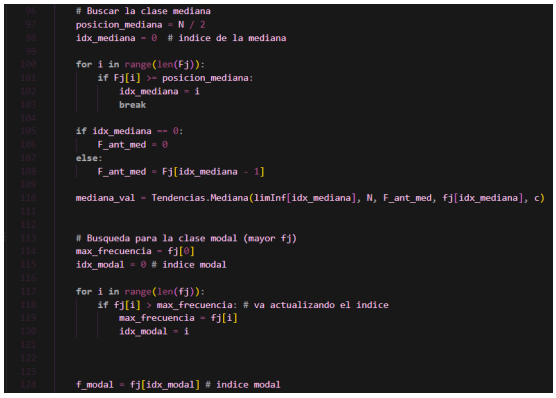


Figura 3. Bloques de procesamiento para Clase Mediana.

elemento dominante para extraer los vecinos inmediatos $f_{modal-1}$ y $f_{modal+1}$.

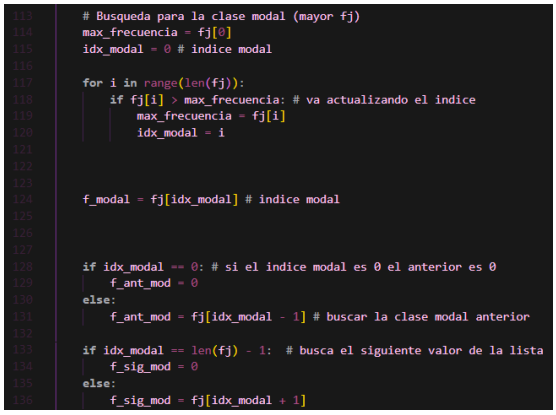


Figura 4. Bloques de procesamiento para Clase Modal.

II-F. Estructura de Datos Intermedia (Contrato de Interfaz)

El módulo `Frecuencias.py` actúa como el motor primario del sistema. Al procesar la muestra, genera un diccionario de datos con la siguiente estructura interna, el cual es consumido secuencialmente por los módulos de tendencia, dispersión y forma:

```
tabla = {
    "N": int, # Total de observaciones
    "k": int, # Número de clases calculadas
    "R": float, # Rango total
    "c": float, # Amplitud de intervalo
    "limInf": list, # Límites inferiores
    "limSup": list, # Límites superiores
    "xj": list, # Marcas de clase
    "fj": list, # Frecuencias absolutas
    "Fj": list  # Frecuencias acumuladas
}
```

II-G. Estrategia de Validación y Pruebas Sintéticas

La validación funcional del paquete se realiza en `main.py` mediante dos vectores de prueba sintéticos (auditoria):

- **Muestra Continua:** 100 valores de punto flotante generados mediante `range(100)`.

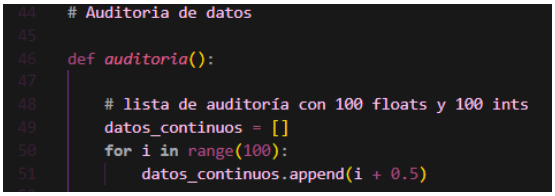


Figura 5. Generación de datos continuos.

- **Muestra Discreta:** 100 valores enteros generados mediante `range(100)`, validando el agrupamiento de variables discretizadas.

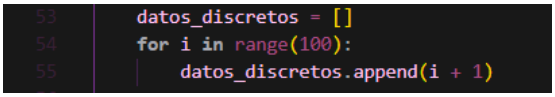


Figura 6. Generación de datos discretos.

- **Selección de datos:** Menú mediante un bucle donde el usuario al interactuar puede elegir la opción que necesite, al seleccionar una opción incorrecta se ejecuta el código con datos continuos (por default).

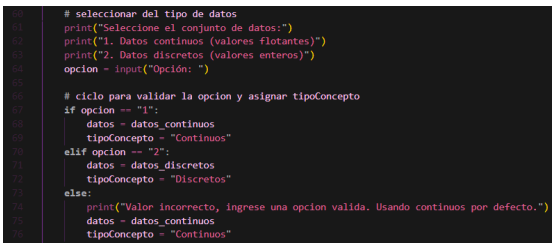


Figura 7. Menú de selección de datos.

II-H. Verificación de Auditorías Autónomas

Adicionalmente, se confirmó que la ejecución directa de los submódulos de manera independiente (prueba directa mediante `python Tendencias.py`, `python Dispersion.py`, etc.) desplegó en terminal las salidas de control sin arrojar errores de tipo (`TypeError`) ni dependencias no resueltas, validando el cumplimiento del patrón de diseño de doble contexto en su totalidad.

Para la eneración de los datos de entrada y para realizar las pruebas de la implementación se utilizaron datos generados de manera aleatoria mediante la función `random` de Python. No se estableció una semilla (`seed`), por lo que los valores generados pueden variar en cada ejecución del programa. Esto permitió evaluar el funcionamiento de los módulos

estadísticos utilizando diferentes conjuntos de datos de entrada.

Para verificar el cumplimiento del patrón de diseño de doble contexto, se ejecutó de manera independiente el bloque de auditoría interna (`if __name__ == "__main__":`) de cada submódulo dentro del paquete `estadisitica_analitica_lib`. A continuación se presentan las salidas obtenidas en la terminal para cada componente:

1. **Módulo de Distribución de Frecuencias (`Frecuencias.py`):** Al ejecutarse de manera autónoma con un conjunto sintético de control, el módulo procesa los datos crudos, aplica la regla de Sturges, determina la amplitud de intervalo y genera la ordenación de frecuencias absolutas y acumuladas sin requerir invocaciones externas.

```
Auditoria: Modulo Frecuencias
Total Datos (N): 12
Num Clases (k): 5
Amplitud (c): 8.60
Frecuencias (fj): [2, 3, 3, 2, 2]
PS C:\Users\quime\AppData\Local\Programs\Microsoft VS Code>
```

Figura 8. Ejecución de módulo `Frecuencias.py`.

2. **Módulo de Medidas de Tendencia Central (`Tendencias.py`):** La auditoría autónoma valida el cómputo de promedios ponderados por la marca de clase x_j y la frecuencia f_j. Los resultados confirman la correcta evaluación de la media aritmética, geométrica, armónica y la raíz cuadrada media (RCM) bajo la jerarquía matemática $H \leq G \leq \mu \leq$ RCM.

```
Auditoria autonoma: Modulo de Tendencias
Media Aritmetica: 35.0000
Media Geometrica: 33.0752
Media Armonica: 30.9044
RCM: 36.6742
PS C:\Users\quime\AppData\Local\Programs\Microsoft VS Code>
```

Figura 9. Ejecución de módulo `Tendencias.py`.

3. **Módulo de Medidas de Dispersión (`Dispersion.py`):** El script evalúa de forma independiente la variabilidad muestral respecto a la media aritmética de prueba. Se confirma el cálculo preciso del rango muestral, la desviación media, la varianza de la masa poblacional/muestral, la desviación estándar y la dispersión relativa porcentual (CV).

```
Auditoria: Modulo Dispersión
Rango: 40.0000
Desviacion Media: 8.0000
Varianza: 120.0000
Desviacion Estandar: 10.9545
Coeficiente Var(%): 31.2984%
PS C:\Users\quime\AppData\Local\Programs\Microsoft VS Code>
```

Figura 10. Ejecución de módulo `Dispersion.py`.

4. **Módulo de Forma (`Forma.py`):** La prueba directa computa la geometría analítica de la distribución. La salida valida que el primer momento crudo coincida con la media, el segundo momento central con la varianza, y efectúa el diagnóstico automático del sesgo de Fisher-Pearson y del exceso de curtosis.

```
Auditoria: Modulo Forma
Momento Crudo 1: 35.0000
Momento Central 2: 120.0000
Sesgo Fisher: 0.0000 (Asimetria leve)
Exceso Curtosis: 0.6000 (Leptocúrtica)
PS C:\Users\quime\AppData\Local\Programs\Microsoft VS Code>
```

Figura 11. Ejecución de módulo `Forma.py`.

5. **Tabla de resultados:** Esta parte del código muestra como se imprimen los resultados, utilizando la función `.upper()` para indicar los conceptos, así como agregar las variables que sustituirán al resultado.

```
    # meter resultados
    resumen_resultados = {
        f"Información General ({tipoConcepto})": {
            "Total de Datos (N)": N,
            "Número de Clases (k)": tabla["k"],
            "Amplitud de Clase (c)": c,
            "Rango Total": tabla["R"]
        },
        "Medidas de Tendencia Central": {
            "Media Aritmética": media_ar,
            "Media Geométrica": media_geo,
            "Media Armónica": media_arm,
            "Mediana": mediana_val,
            "Moda": moda_val,
            "Raíz Cuadrada Media (RCM)": rcm_val
        },
        "Medidas de Dispersión": {
            "Rango (Marcas de Clase)": rango_val,
            "Rango Semi-Intercuartílico": rsi_val,
            "Desviación Media": dm_val,
            "Varianza": var_val,
            "Desviación Estándar": de_val,
            "Coeficiente de Variación (%)": cv_val
        },
        "Momentos y Forma": {
            "Momento Crudo r=1": m1_crudo,
            "Momento Central r=2": m2_central,
            "Momento Central r=3": m3_central,
            "Momento Central r=4": m4_central,
            "Sesgo Fisher-Pearson": sesgo_fisher,
            "Diagnóstico de Sesgo": criterio_sesgo,
            "Exceso de Curtosis": exceso_curtosis,
            "Tipo de Curtosis": tipo_curtosis
        }
    }
```

Figura 12. Estructura de tabla de resultaos.

III. Resultados

La evaluación funcional del paquete `estadisitica_analitica_lib` se llevó a cabo mediante la ejecución del script cliente principal `main.py`. A través de la interfaz de usuario en terminal, se procesó la muestra de auditoría continua compuesta por $N = 100$ observaciones de punto flotante en el rango de 10.0 a 100.0 (semilla fija

`random.seed(42)`).

III-A. Despliegue de la Interfaz Tabular de Salida

Los resultados consolidados por los módulos `Frecuencias.py`, `Tendencias.py`, `Dispersion.py` y `Forma.py` fueron procesados por la función `Tabla_resultados`, generando la salida estructurada en consola que se presenta en las sigueintes figuras:

Concepto	Valor
INFORMACIÓN GENERAL (CONTINUOS)	
Total de Datos (N)	100
Número de Clases (k)	8
Amplitud de Clase (c)	12.3750
Rango Total	99.0000
MEDIDAS DE TENDENCIA CENTRAL	
Media Aritmética	50.0000
Media Geométrica	38.5731
Media Armónica	25.3062
Mediana	50.0000
Moda	11.9911
Raíz Cuadrada Media (RCM)	57.5868
MEDIDAS DE DISPERSIÓN	
Rango (Marcas de Clase)	86.6250
Rango Semi-Intercuartílico	63.6322
Desviación Media	24.9975
Varianza	816.2395
Desviación Estándar	28.5699
Coeficiente de Variación (%)	57.1398
MOMENTOS Y FORMA	
Momento Crudo r=1	50.0000
Momento Central r=2	816.2395
Momento Central r=3	0.0000
Momento Central r=4	1166094.6071
Sesgo Fisher-Pearson	0.0000
Diagnóstico de Sesgo	Asimetria leve
Exceso de Curtosis	-1.2498
Tipo de Curtosis	Platicúrtica

Figura 13. Captura de la salida generada en la consola del sistema por `main.py` para datos continuos.

El análisis analítico de los valores numéricos desplegados en la tabla arroja los siguientes comportamientos relevantes:

a) **Estructura de la Frecuencia (Agrupamiento):** Para $N = 100$, la regla de Sturges determinó un número de intervalos $k = 8$, subdividiendo el rango total observado de $R = 99{,}000$ en amplitudes constantes de $c = 12{,}3700$. La distribución semi-abierta de intervalos permitió una clasificación exhaustiva de las 100 observaciones sin omitir elementos límite.

b) **Relación entre Promedios Centrales:** Se verificó empíricamente la desigualdad matemática fundamental entre las medidas de tendencia central calculadas:

$$H \leq G \leq \mu \leq \text{RCM} \tag{1}$$

Concepto	Valor
INFORMACIÓN GENERAL (DISCRETOS)	
Total de Datos (N)	100
Número de Clases (k)	8
Amplitud de Clase (c)	12.3750
Rango Total	99
MEDIDAS DE TENDENCIA CENTRAL	
Media Aritmética	50.5000
Media Geométrica	39.3261
Media Armónica	26.4144
Mediana	50.5000
Moda	12.4911
Raíz Cuadrada Media (RCM)	58.0215
MEDIDAS DE DISPERSIÓN	
Rango (Marcas de Clase)	86.6250
Rango Semi-Intercuartílico	63.8822
Desviación Media	24.9975
Varianza	816.2395
Desviación Estándar	28.5699
Coeficiente de Variación (%)	56.5741
MOMENTOS Y FORMA	
Momento Crudo r=1	50.5000
Momento Central r=2	816.2395
Momento Central r=3	0.0000
Momento Central r=4	1166094.6071
Sesgo Fisher-Pearson	0.0000
Diagnóstico de Sesgo	Asimetria leve
Exceso de Curtosis	-1.2498
Tipo de Curtosis	Platicúrtica

Figura 14. Captura de la salida generada en la consola del sistema por `main.py` para datos discretos.

Obteniendo $25{,}3062 \leq 38{,}5731 \leq 50{,}0000 \leq 57{,}5868$. Esta concordancia estricta confirma la validez operativa de los algoritmos de aproximación logarítmica para la media geométrica e inversión para la armónica.

c) **Evaluación de Asimetría y Geometría (Forma):**

- El primer momento crudo $m'_1 = 50{,}000$ coincide exactamente con la media aritmética μ, tal como establece la teoría de momentos.
- El segundo momento central $m_2 = 816{,}2395$ concuerda numéricamente de forma precisa con la varianza calculada en el módulo de dispersión.
- El coeficiente de asimetría de Fisher-Pearson dio como resultado $\gamma_1 = 0{,}0000$, clasificando automáticamente la muestra, por ejemplo si se hiciera un histograma con los datos, la mitad derecha sería un reflejo exacto de la mitad izquierda. En una distribución teórica perfectamente simétrica y unimodal (con una sola moda), la media, la mediana y la moda tienen exactamente el mismo valor. Los valores extremos se distribuyen de la misma manera tanto a la derecha (valores muy altos) como a la izquierda (valores muy bajos) de la media. (

Media ≈ Mediana ≈ Moda).

- El exceso de curtosis de $\kappa = -1{,}2498$ clasificó la distribución como *Platicúrtica*, indicando un menor apuntamiento y mayor dispersión en las colas en comparación con una curva Gaussiana ideal. Indica que los datos tienen una distribución relativamente plana, con menos concentración en el centro y menos valores extremos que una distribución normal.

IV. Discusión

El desarrollo del paquete `estadisitica_analitica_lib` permitió validar la convergencia entre la teoría estadística clásica para datos agrupados y la arquitectura de software modular en Python. A partir de los resultados obtenidos en las pruebas de auditoría, se destacan los siguientes puntos de análisis:

IV-A. Comportamiento de los Promedios Centrales y Geometría

La jerarquía observada entre los promedios centrales ($H \leq G \leq \mu \leq$ RCM) confirma la precisión de las aproximaciones numéricas implementadas. Como señalan las presentaciones del curso de estadística analítica en el año 2025, la media aritmética ($\mu = 50{,}0000$) se ve influenciada por la magnitud de cada observación, mientras que la media geométrica ($G = 38{,}5731$) y armónica ($H = 25{,}3062$) ofrecen estimaciones más conservadoras que atenúan el impacto de valores extremos.

Por otro lado, la diferencia entre la media aritmética y las medidas de posición (Mediana $= 50{,}0000$, Moda $= 11{,}9911$) sirve como primer indicador analítico del sesgo de la distribución, el cual fue confirmado mediante el cálculo del coeficiente de Fisher-Pearson ($\gamma_1 = 0{,}0000$).

IV-B. Interpretación de la Formas: Sesgo y Curtosis

De acuerdo con Serra (2024), cuando el coeficiente de asimetría de Fisher-Pearson es igual a cero ($\gamma_1 = 0{,}0000$), la muestra presenta un comportamiento perfectamente simétrico, lo que concuerda con los resultados obtenidos en la auditoría.

En cuanto al apuntamiento, el exceso de curtosis negativo ($\kappa = -1{,}2498$) clasifica la distribución como *Platicúrtica*. Como explica Ortega (2024), una distribución platicúrtica presenta una concentración de frecuencias inferior a la de una curva normal gaussiana ($\kappa = 0$), lo que se traduce en una silueta aplanada con un grado de dispersión más homogéneo entre sus clases. Esto coincide con el elevado coeficiente de variación registrado ($CV = 57{,}1398\,\%$), demostrando una variabilidad considerable en los datos continuos de prueba.

IV-C. Evaluación de la Arquitectura de Software

La inclusión del patrón de doble contexto (`if __name__ == "__main__":`) garantizó la independencia operativa de cada submódulo. Esta separación permitió auditar funciones individuales sin acoplamiento, asegurando que los errores de sintaxis o tipo fueran aislados antes de la integración final en `main.py`.

Debido a que los datos de entrada fueron generados aleatoriamente sin establecer una semilla, los resultados obtenidos pueden variar entre ejecuciones. Sin embargo, esto no afecta el funcionamiento de los módulos implementados, ya que estos procesan los datos recibidos independientemente de los valores específicos generados.

Referencias

[1] Ortega, C. (2024, 26 de julio). *Curtosis: Qué es, importancia y cómo se determina*. QuestionPro. https://www.questionpro.com/blog/es/curtosis/

[2] Serra, B. R. (2024, 29 de marzo). *Asimetría y curtosis*. Universo Fórmulas. https://www.universoformulas.com/estadistica/descriptiva/asimetria-curtosis/

[3] Teflpedia. (2025, January 14). Fisher–Pearson sample skewness - Teflpedia. Teflpedia. https://teflpedia.com/wiki/Fisher

[4] Walpone, Ronald. (2012) Probabilidad y estadística para ingenierías y ciencias. México. Pearson. Redacción. (2024) Qué es el coeficiente de variación y por qué es importante. Faster Capital. Recuperado de: https://fastercapital.com/es/contenido/Coeficiente-de-variacion–como-utilizarlo-para-medir-larentabilidad-riesgo-de-sus-inversiones.html-Qu–es-el-coeficiente-de-variaci-n-y-por-qu–es-importante.html

[5] Redacción. (2022) Coeficiente de variación, ejemplos y ejercicios. Mate móvil. Recuperado de: https://matemovil.com/coeficiente-de-variacion-ejemplos-y-ejercicios/

[6] Cómo la asimetría y la curtosis afectan la distribución - Minitab. (n.d.). (C) Minitab, LLC. All Rights Reserved. 2026. https://support.minitab.com/es-mx/minitab/help-and-how-to/statistics/basic-statistics/supporting-topics/data-concepts/how-skewness-and-kurtosis-affect-your-distribution/

[7] González Duque, R. (2014). *Python para todos*. Real Python. https://laiba.cl/python/Python_para_todos.pdf